Tópicos avanzados de cómputo

Desarrollo Backend

-

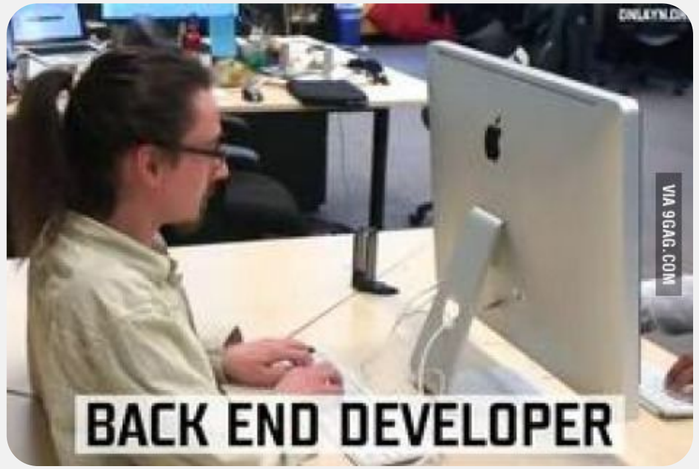
Profesor: Rafael Pérez Aguirre



¿Qué es *backend*?

Backend (o *server-side*) es la parte de un sistema o aplicación que se **ejecuta en el servidor**, no es visible para el usuario y **gestiona la lógica de negocio** y el **acceso a datos** (p. ej., bases de datos y servicios/API) para responder a las peticiones del frontend.

Es la cocina de tu app: donde viven la lógica, los datos y las integraciones que hacen posible la experiencia que el usuario ve en el frontend.

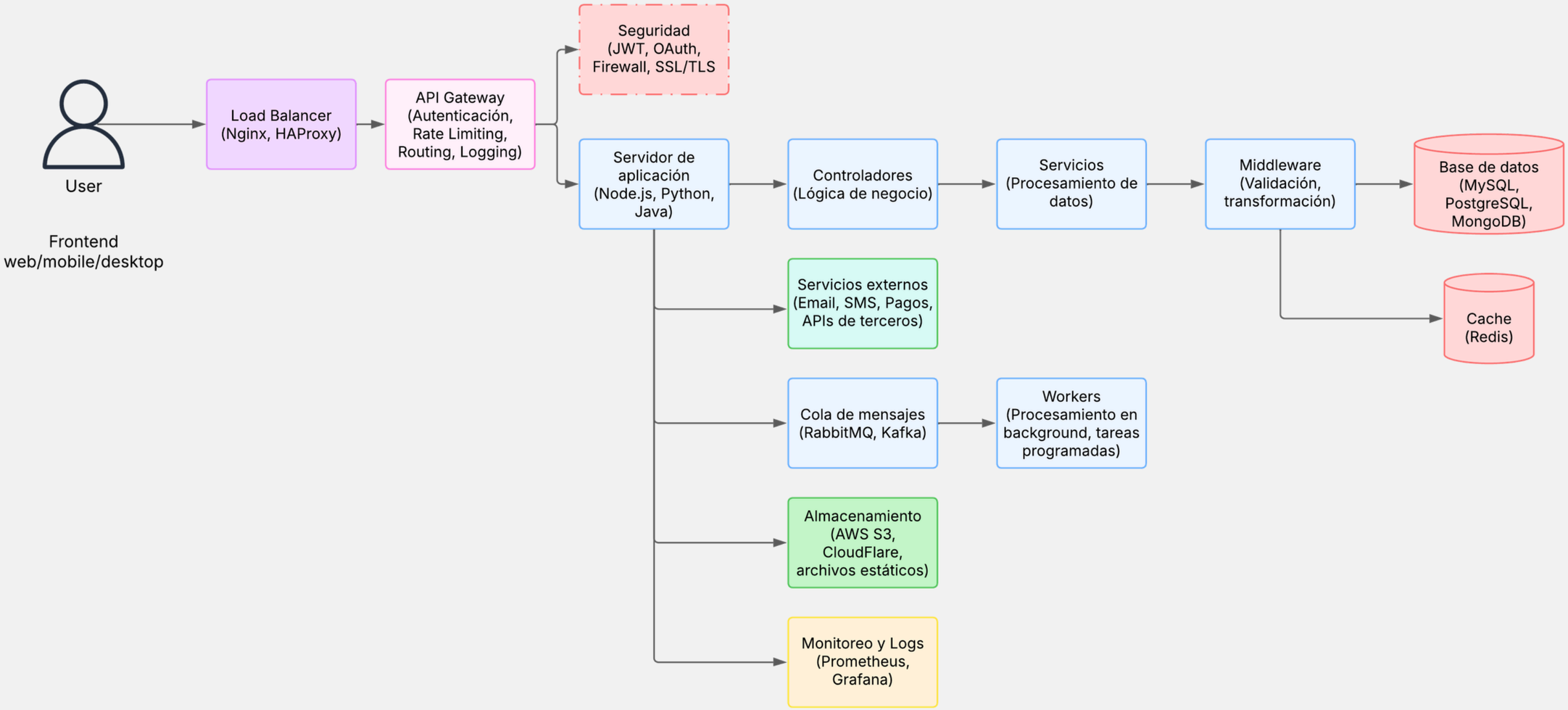


Qué hace el backend



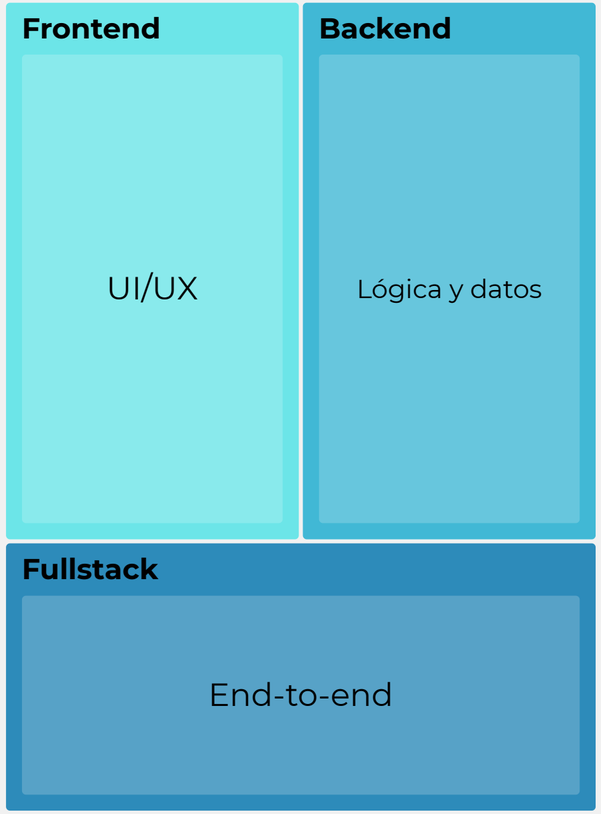
- Expone **APIs** (REST/GraphQL) consumidas por web, móvil e IoT.
- Aplica **reglas de negocio** y valida entradas (integridad, formatos, límites).
- **Persiste** y **consulta** datos (SQL/NoSQL), usa **cachés** y **colas**.
- Se integra con **servicios externos**: pagos, correo, notificaciones, almacenamiento.
- Implementa **seguridad**: autenticación (JWT/OAuth2), autorización (RBAC/ABAC), rate limiting.
- Garantiza **confiabilidad** y **observabilidad**: logs estructurados, métricas, trazas, health checks.
- Orquesta **despliegue** y **operación**: contenedores, CI/CD, rollbacks, cloud/on-premise

Arquitectura de backend



Frontend vs Backend vs Fullstack

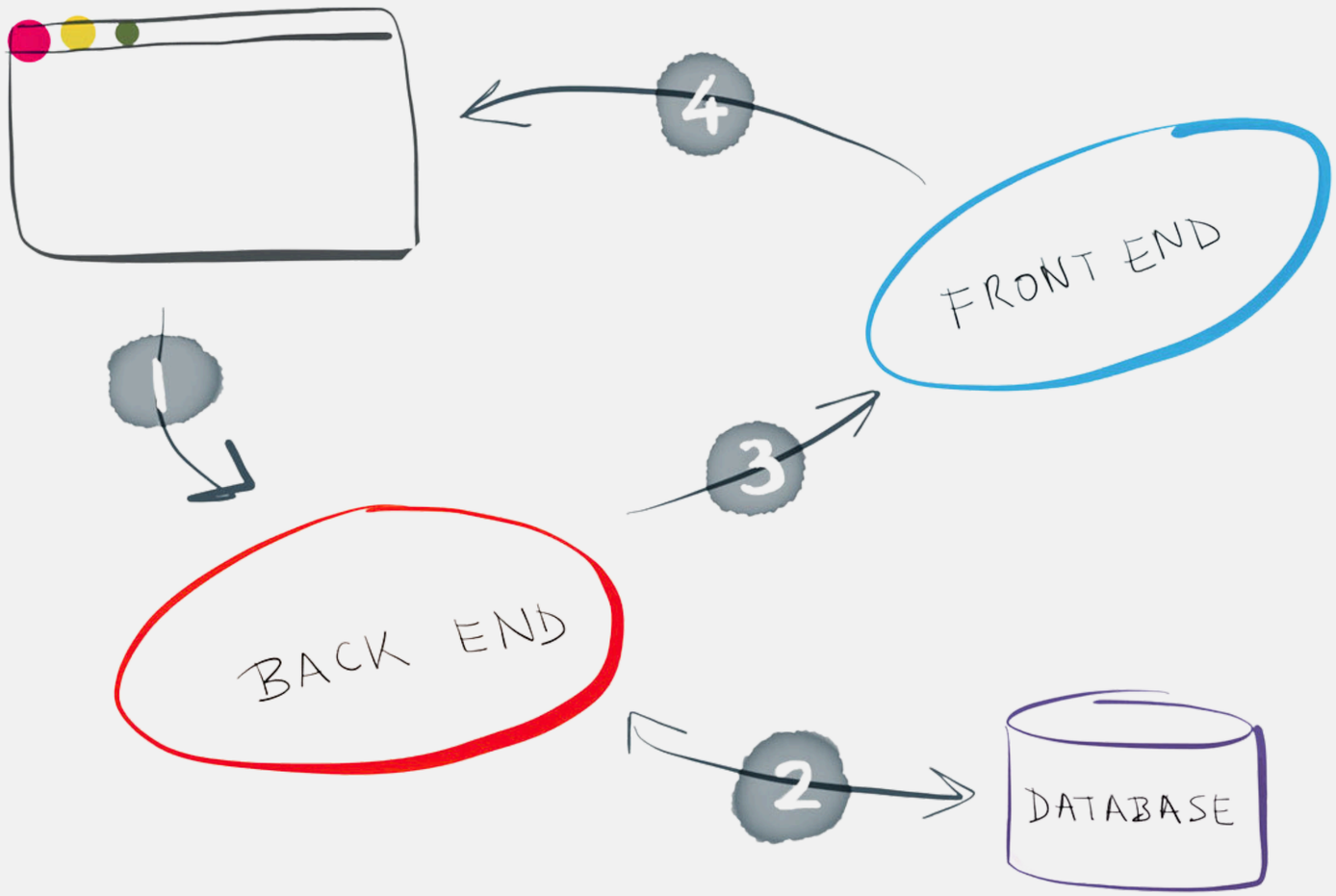
- **Frontend:** interfaz, interacción y experiencia del usuario.
- **Backend:** lógica de negocio, datos, integraciones y operación.
- **Fullstack:** cubre extremo a extremo, priorizando entregas verticales.



Comparativa

Rol	Enfoque	Responsabilidades clave	Entregables / artefactos	Tecnologías
Frontend	UI/UX	Componentes, estados, accesibilidad, routing, consumo de APIs	Prototipos, componentes, tests de UI, bundle	HTML/CSS/JS, React/Vue/Svelte, Vite/Webpack, PWAs
Backend	Lógica y datos	Diseño de APIs, validación, seguridad, persistencia, colas, cache, observabilidad	Contratos (OpenAPI), endpoints, migraciones, jobs, tests	Python (Flask/FastAPI), Node (Express/Nest), SQL/NoSQL, Redis, RabbitMQ/Kafka, Docker
Fullstack	End-to-end	Entregas verticales (UI+API+DB), CI/CD, IaC básica	Feature completo, PRs, pipelines	Stack mixto anterior + frameworks fullstack (Next.js/Nuxt)

Ejemplo



Rol del desarrollador backend

Es responsable de:

- **Diseñar y versionar APIs** (REST/GraphQL) con contratos claros.
- **Modelar el dominio y los datos** (tablas, índices, relaciones; SQL/NoSQL).
- **Aplicar reglas de negocio** y validar entradas/salidas (esquemas).
- **Seguridad:** autenticación (JWT/OAuth2) y autorización (RBAC/ABAC).
- **Rendimiento y escalabilidad:** cachés, colas, horizontal scaling, backpressure.
- **Resiliencia:** timeouts, retries, circuit breakers, rate limiting.
- **Observabilidad:** logs estructurados, métricas, tracing, health checks.
- **Integraciones:** pagos, correo, almacenamiento, terceros; webhooks.
- **Operación:** CI/CD, contenedores, configuración por entorno, rollbacks.

Stack

- Javascript, Python, Java
- HTTP/REST, JSON, OpenAPI
- SQL (JOINs, índices, transacciones/ACID)
- NoSQL
- ORM
- Redis
- Colas (RabbitMQ/Kafka)
- Pruebas (unitarias/integración/contract tests)
- Docker
- Cloud (red, IAM, storage)
- CLI
- ML















Comunicación, escritura técnica, estimación y scoping, code reviews, ética profesional (datos y privacidad)

Monolito vs. Microservicios

Monolito

Toda la lógica vive en un mismo proyecto y se despliega como una sola unidad.

VENTAJA

Simple al inicio · fácil de depurar · sin infraestructura compleja

DESVENTAJA

Escalar una parte obliga a escalar todo · redeploy completo por cualquier cambio

Microservicios

Servicios independientes (usuarios, pagos, catálogo…) que se comunican por API.

VENTAJA

Cada equipo escala y despliega su servicio de forma independiente

DESVENTAJA

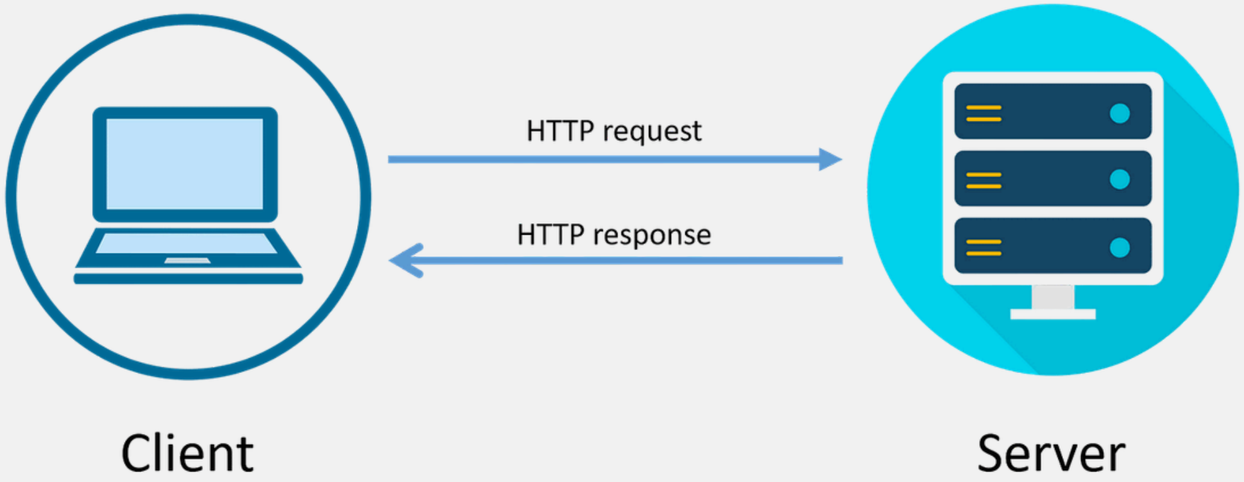
Más complejidad operativa: orquestar, monitorear, tolerar fallos parciales

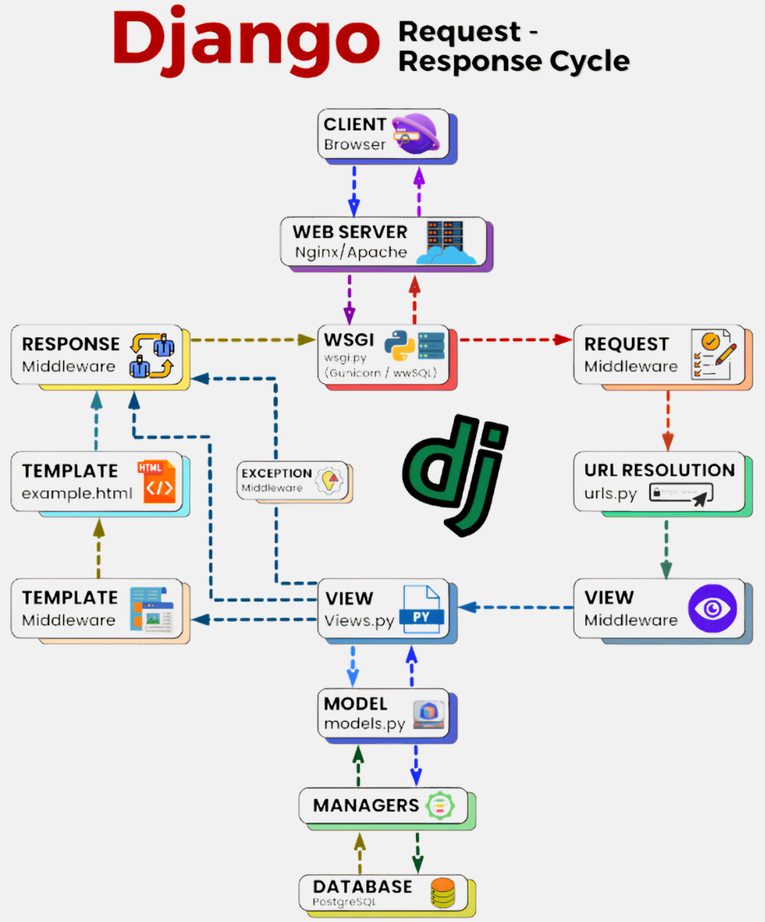
¿Cuándo conviene cada arquitectura?

Para un proyecto nuevo, un MVP o un equipo pequeño, **un monolito bien organizado casi siempre es la decisión correcta**, es más rápido de construir y más barato de operar.

Los microservicios se justifican cuando el equipo y el producto ya crecieron lo suficiente para que el costo operativo valga la pena.

El ciclo request–response





HTTP

HTTP (*Hypertext Transfer Protocol*) es un **protocolo de aplicación**, **sin estado**, de tipo cliente-servidor, que define la **semántica de peticiones y respuestas** (métodos como GET/POST, cabeceras y códigos de estado) para **transferir representaciones de recursos identificados por URI** en la Web.



Sus semánticas son independientes del transporte:
hoy se usa sobre TCP (p. ej., HTTP/1.1, HTTP/2)

Estructura básica



- **Request** = método + URL + headers + (opcional) body.
- **Response** = status code + headers + (opcional) body.

Métodos (semántica)

- **GET**: lectura.
- **HEAD:** como GET, sin body.
- **POST**: crear/accionar.
- **PUT**: reemplazo completo.
- **PATCH**: actualización parcial.
- **DELETE**: borrar.
- **OPTIONS:** capacidades/CORS.



Visión API First

El contrato de la API (qué endpoints existen, qué reciben, qué regresan) se diseña y documenta ***antes*** de escribir la implementación.

CODE FIRST

1. Se escribe el backend
2. El equipo de frontend espera
3. “El API ya está listo”
4. Se documenta al final (si acaso)

API FIRST

1. Se diseña el contrato (OpenAPI)
2. Frontend construye contra un mock
3. Backend implementa en paralelo
4. Documentación y contrato: una sola fuente